

Anime Discovery, Recommendation, and Graph Intelligence

Alvaro Manuel De Cossio Velasquez | U20221F812

Project Proposal

This project builds a reproducible anime discovery and recommendation system. The domain is anime catalog intelligence: the system combines content metadata, user ratings, user recommendation links, and franchise relation edges to support discovery.

The product question is:

Which anime are similar, which latent segments exist in the catalog, and which titles should be candidates for recommendation or graph-based ranking?

The project fits the course because it is not a one-table prediction exercise. It has a catalog layer, feature layer, interaction layer, graph layer, and executable pipeline. By Week 7, the project can support representation learning, clustering, recommendation candidate generation, and later graph-aware reranking.

Source Inventory

The dataset combines public API data, public cache data, and a Kaggle interaction dataset. AniDB live calls are rate-limited and cached; Shoko XML files are used as a bulk cache seed to avoid unnecessary live requests. Source URLs are listed in the references.

Table 1. Source Inventory, Access Conditions, and Provenance

Source	Access path	Format	Access condition	Project use
Jikan	Public HTTP API	JSON	Public API with request throttling	MAL catalog metadata, relations, recommendations
MAL ID cache	GitHub public cache	JSON	Public repository cache	MAL-to-AniDB identifier matching
AniDB	HTTP XML API	XML	Public API; cached to reduce live calls	Live episode and metadata repair
Shoko cache	Public XML archive	XML zip	Public cache archive	Bulk AniDB metadata cache seeding
Kaggle ratings	Kaggle dataset	CSV	Kaggle dataset page terms	User-item interaction layer

Final source scale is non-trivial: the processed anime catalog contains **16,664 rows** and **32 columns**, while the aligned ratings table contains **148,170,496 user-item ratings**. The large ratings file is processed through scripts rather than manually loaded into a notebook.

Table 2. Week 3 Scale and Quality Summary

Check	Result
Catalog scale	16,664 anime rows, 32 columns
Ratings scale	148,170,496 user-item ratings
Primary key	mal_id; one row per MAL anime title
Missingness	domain-aware checks reported in EDA figure
Subset rule	valid anime formats only; invalid IDs logged separately
Memory strategy	large ratings table processed by scripts and chunks

Pipeline and Dataset Build

The build starts from candidate MAL IDs and ratings data. For each anime candidate, Jikan provides the main MAL metadata. The MAL ID cache maps available MAL IDs to AniDB IDs. AniDB metadata is first read from the local Shoko-seeded cache; live AniDB calls are reserved for missing or stale fields such as current episode counts and missing metadata. Failed API calls are recorded in retry registries, while permanent invalid IDs are recorded separately.

The numbered scripts now follow the same order as the project workflow:

```
python src/01_run_dataset_ingestion.py
python src/02_run_ratings_ingestion.py
python src/03_build_catalog_features.py
python src/04_apply_dataset_improvements.py
python src/05_run_week5_dimensionality.py
python src/06_run_week7_clustering.py
python src/07_build_recommendation_graph.py
python src/09_create_project_visualizations.py
```

Table 3. Reproducible Outputs Used in Weeks 3–7

Artifact	Purpose
data/processed/anime_dataset.csv	Main anime catalog
data/processed/anime_dataset.json	JSON copy of catalog
data/processed/ratings_processed.csv	Aligned ratings interaction table
data/caches/anidb_metadata_cache.json	Cached AniDB metadata
data/build/failed_api_requests.json	Retry registry for failed API calls
artifacts/plots/	Generated EDA and milestone figures

Schema Draft and Data Dictionary

The main catalog grain is one row per MAL anime ID. This key is used for joins with ratings and for parsing item-item edges from recommendations and relations.

Table 4. Core Anime Catalog Fields

Column	Type	Meaning and quality note
mal_id	integer	Primary key from MyAnimeList
anidb_id	integer	External AniDB identifier when available
title	text	Main title
type	categorical	TV, Movie, OVA, ONA, Special, TV Special
episodes	numeric	Episode count; repaired from AniDB when possible
duration	numeric	Minutes per episode or title
total_watch_minutes	numeric	Episodes multiplied by duration
score, members, favorites	numeric	Rating and popularity signals
genres	multi-label	MAL genre labels, with gap filling from AniDB tags
tags	multi-label	MAL themes plus AniDB non-explicit tags
explicit_tags	multi-label	Explicit AniDB tag branch labels
demographics	multi-label	Audience labels normalized across MAL and AniDB
studios	multi-label	Studio labels, with AniDB animation-work fallback
relations	edge list	Typed anime-anime story/franchise edges
recommendations	weighted edge list	User recommendation edges from MAL/Jikan

The interaction table uses `ratings_processed.animeID = anime_dataset.mal_id`. This creates a clean user-item layer without depending on the original Kaggle internal anime identifier.

Processed Dataset V3 and Exploratory Analysis

Week 3 required a first processed dataset, so the project first produced a cleaned catalog table that could be regenerated from the ingestion command. The current processed dataset is the later post-retry and post-improvement version. It removes negligible rows without date or episode information, normalizes seasons from aired month, converts duration to numeric minutes, and adds total watch time. Remaining gaps are documented rather than hidden.

The first EDA check is format balance. This matters because TV series, movies, OVAs, ONAs, Specials, and TV Specials have different episode and runtime distributions. A recommender should not treat a 12-episode TV season and a 90-minute movie as interchangeable simply because both are anime.

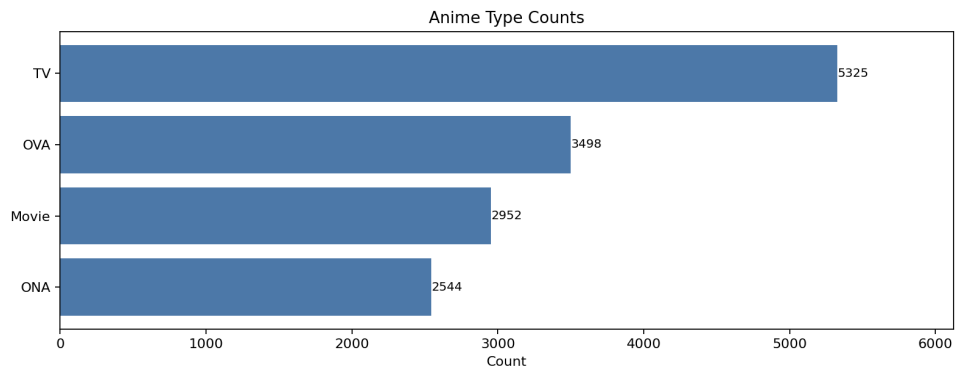


Figure 1. Anime type distribution after dataset improvements

The second EDA check is genre/tag coverage. Top genres provide a sanity check that the catalog still represents mainstream discovery categories after API filtering and AniDB enrichment.

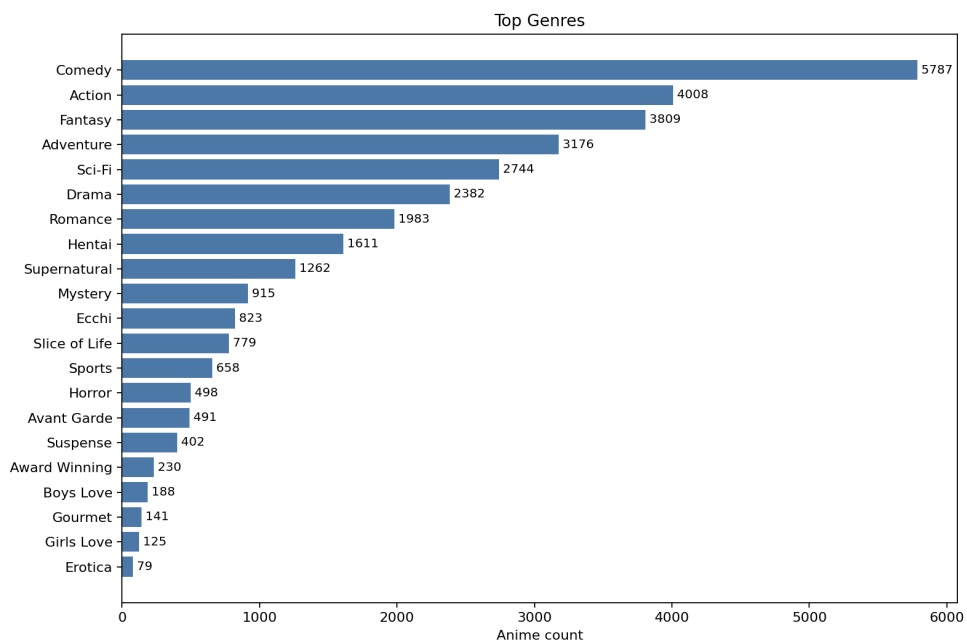


Figure 2. Most frequent genre labels in the processed catalog

Missingness is treated with domain logic. Empty explicit tags are not considered a problem for non-explicit titles; duration and demographics are more meaningful remaining gaps. This prevents the report from overstating missingness in fields where an empty value can be valid.

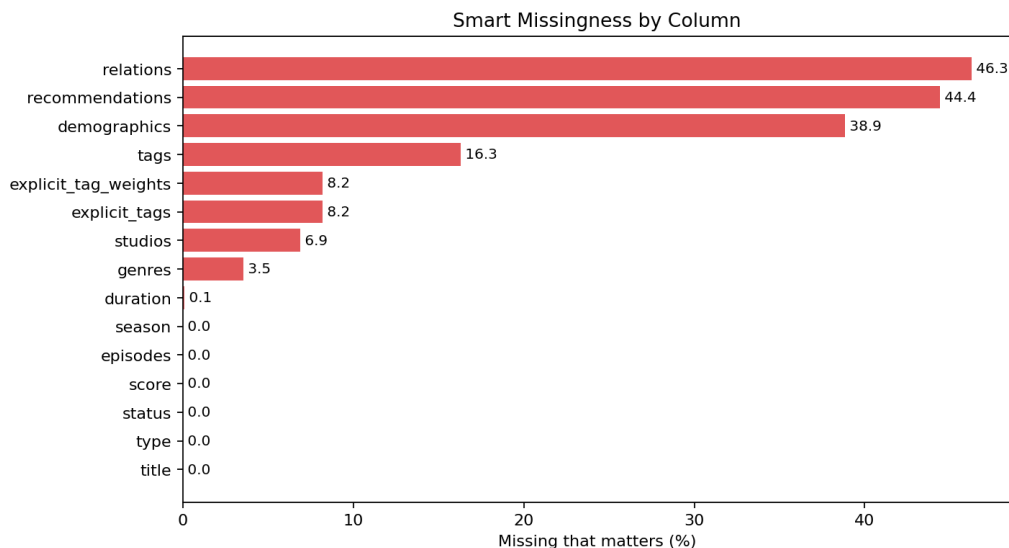


Figure 3. Smart missingness after domain-aware checks

The ratings table is too large to inspect by loading it all into memory. The EDA notebook uses chunked processing to count ratings per user and per anime. This validates that the interaction layer has enough repeated behavior for recommendation work and also reveals sparsity.

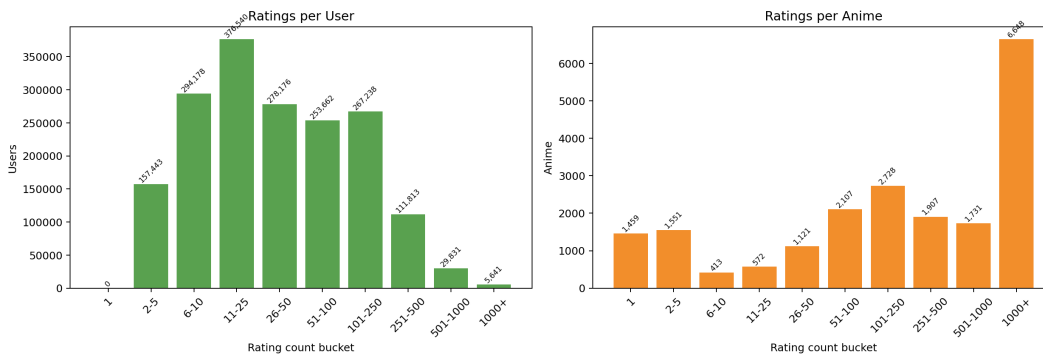


Figure 4. Quantity distribution of ratings per user and per anime

Graph Layer

The graph layer has two edge families. Recommendation edges are directed and weighted by MAL recommendation votes, encoded as `target_mal_id:votes`. Relation edges are typed story or franchise edges such as Sequel, Prequel, Side Story, Parent Story, Alternative Version, and Summary.

Inside the final catalog, the parsed graph contains **114,523 recommendation edges** and **14,392 relation edges**. Recommendation edges represent user-perceived similarity, while relation edges represent franchise navigation.

Recommendation Network Among Top-Popularity Anime

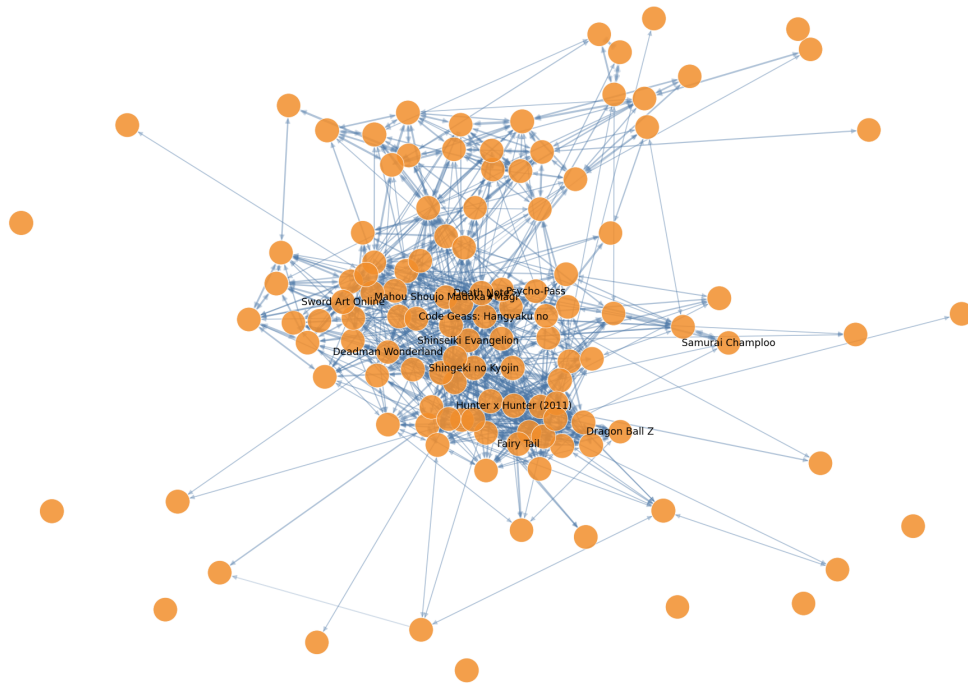


Figure 5. Readable subgraph of recommendation connections among popular titles

The Bakemonogatari example shows why relation edges matter. A recommender should know that connected entries exist, but relation type alone is not a universal watch order. Some prequels are released later, some specials are optional, and large franchises may have multiple valid entry points.

Bakemonogatari Relation Tree: Franchise Connectivity, Not Final Watch Order



Figure 6. Bakemonogatari relation tree showing franchise connectivity, not final watch order

Ethics and Access Note

This project uses public APIs, public cache files, and a Kaggle dataset. It does not scrape private MAL, Instagram, or LinkedIn data. The ratings dataset uses anonymous numeric user identifiers. Processed outputs do not contain names, emails, profile URLs, or private user attributes. API use is rate-limited and cached to reduce unnecessary traffic. The main data risk is behavioral re-identification from ratings patterns, so the project does not redistribute raw personal profiles or attempt user identity reconstruction.

Week 5: Representation and Dimensionality

The Week 5 representation combines numeric, categorical, text, and tag fields. PCA is applied to the full anime row set through a dense mixed catalog matrix. The TF-IDF vocabulary is capped because PCA in scikit-learn requires dense input. Truncated SVD is applied to the larger sparse text/tag matrix because it preserves the sparse representation without a large dense conversion.

The reason for using two matrices is practical and methodological. The dense matrix answers the instructor's question about applying PCA to the full dataset: every processed anime row is represented, but the text vocabulary is intentionally compact so that PCA can run on dense data. The sparse matrix keeps the richer text and tag signal, and SVD is used because it is the standard PCA-like method for sparse TF-IDF data. This gives a fair comparison instead of forcing one method onto an unsuitable representation.

Table 5. Feature Representation Design

Block	Representation	Reason
Numeric	standardized dense values	scores, popularity, dates, runtime
Categorical	multi-hot encoding	type, rating, season, genres, demographics
Text/tags	TF-IDF	synopsis, tags, explicit tags, studios
Graph-adjacent	edge counts and relation fields	supports franchise and recommendation analysis

The output table below is the main Week 5 compression result. PCA reaches 90 percent retained variance with 25 components and 95 percent with 43 components on the dense mixed catalog. SVD reaches 90 percent retained energy with 27 components and 95 percent with 73 components on the sparse catalog. The similar component counts at 90 percent suggest that both representations contain strong redundancy, but the sparse representation needs more components near 95 percent because text and tag features preserve many niche distinctions.

Table 6. Dimensionality Reduction Output Table

Method	Target	Components needed	Max tested	Retained at max
PCA dense mixed catalog	0.50	4	80	0.968
PCA dense mixed catalog	0.70	8	80	0.968
PCA dense mixed catalog	0.80	14	80	0.968
PCA dense mixed catalog	0.90	25	80	0.968
PCA dense mixed catalog	0.95	43	80	0.968
TruncatedSVD sparse full catalog	0.50	5	120	0.955
TruncatedSVD sparse full catalog	0.70	9	120	0.955
TruncatedSVD sparse full catalog	0.80	15	120	0.955
TruncatedSVD sparse full catalog	0.90	27	120	0.955
TruncatedSVD sparse full catalog	0.95	73	120	0.955

The PCA curve evaluates how much variance is retained by the dense mixed representation.

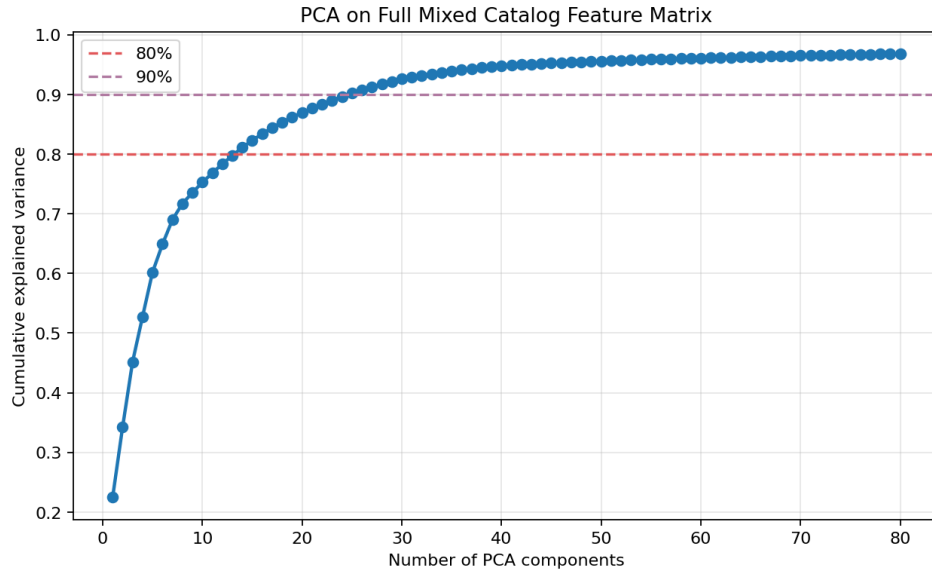


Figure 7. PCA cumulative explained variance on the dense mixed catalog matrix

The first two PCA components are shown only as a diagnostic. They help identify whether broad format and popularity effects dominate the dense representation.

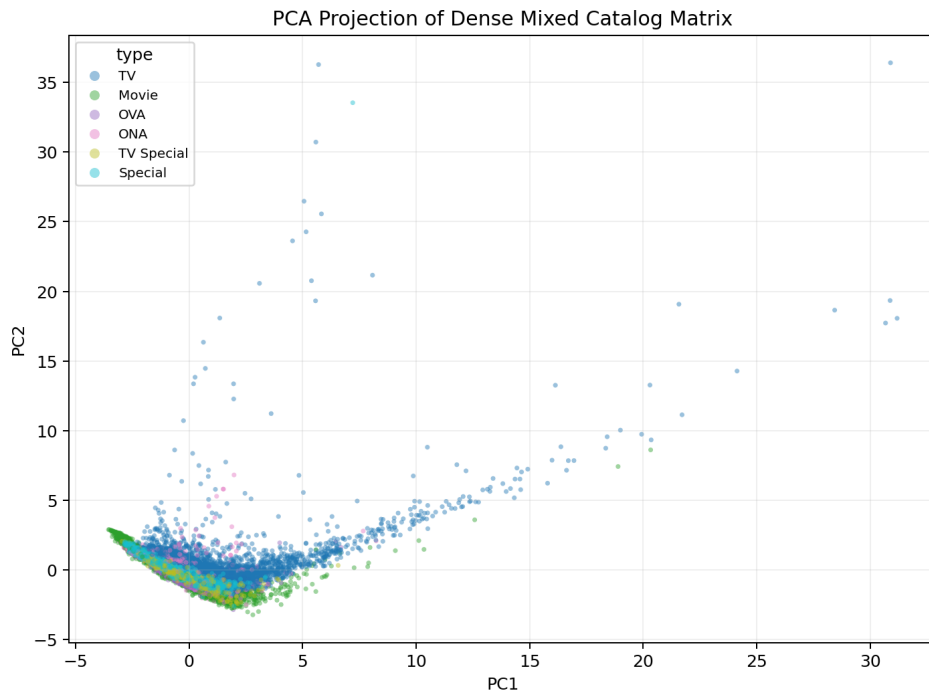


Figure 8. First two PCA components colored by anime type

For the sparse text/tag representation, SVD is the more appropriate linear method. The singular value decay shows how concentrated the sparse representation is.

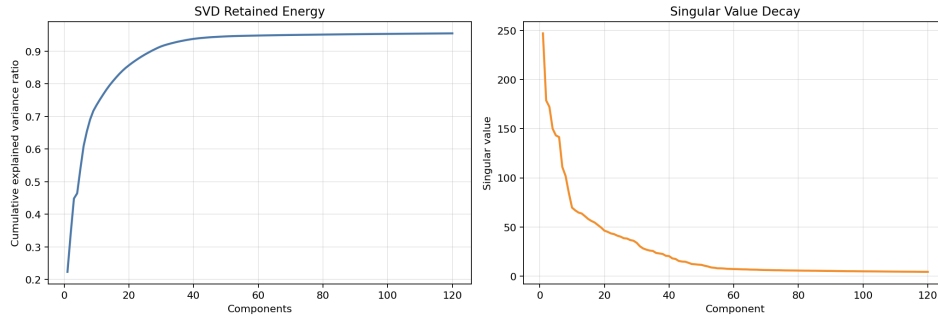


Figure 9. SVD retained energy and singular value decay on the sparse catalog matrix

t-SNE is included for visual comparison on a **6,000-title sample** of SVD embeddings. It is not used as a production feature space and it is not treated as cluster proof.

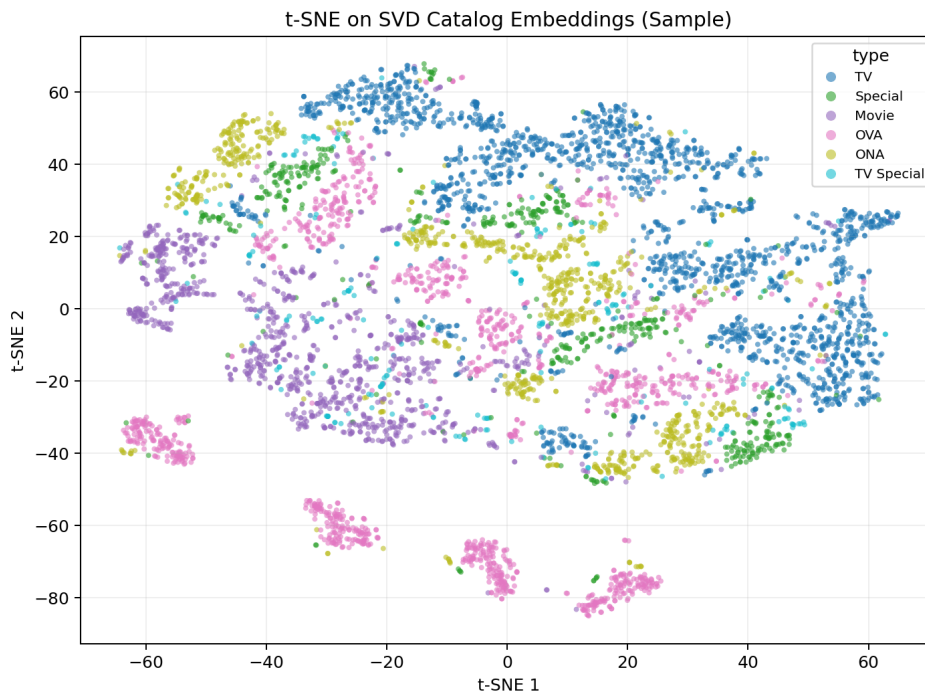


Figure 10. t-SNE visualization of a sample of SVD catalog embeddings

Week 7: Clustering and Validation

Clustering is performed on an SVD-compressed mixed representation. K-means, DBSCAN, and OPTICS are compared because they make different assumptions about shape, density, and outliers.

K-means is tested over multiple values of k . It assumes Euclidean, roughly spherical clusters and assigns every anime to a segment. In the newest run, $k = 20$ has the best silhouette among tested K-means settings, with silhouette about **0.104**. This is still low, so the interpretation is cautious: K-means is useful as a broad segmentation tool, not as proof that anime genres form clean separated islands.

Table 7. K-means Parameter Sweep Output

k	Silhouette	Davies-Bouldin	Inertia
4	0.057	3.933	780172.8
6	0.029	3.699	748353.9
8	0.050	3.350	716880.6
10	0.051	2.891	685516.7
12	0.073	2.840	664545.4
14	0.057	2.593	641766.3
16	0.083	2.479	618528.1
18	0.076	2.423	597438.6
20	0.104	2.389	576681.7

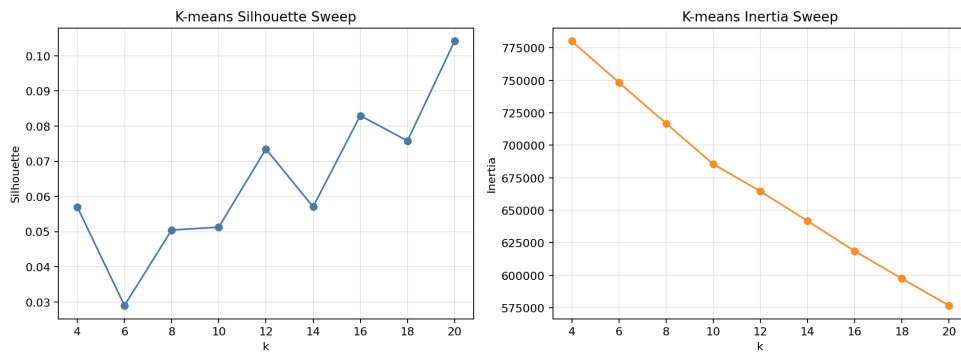


Figure 11. K-means silhouette and inertia parameter sweep

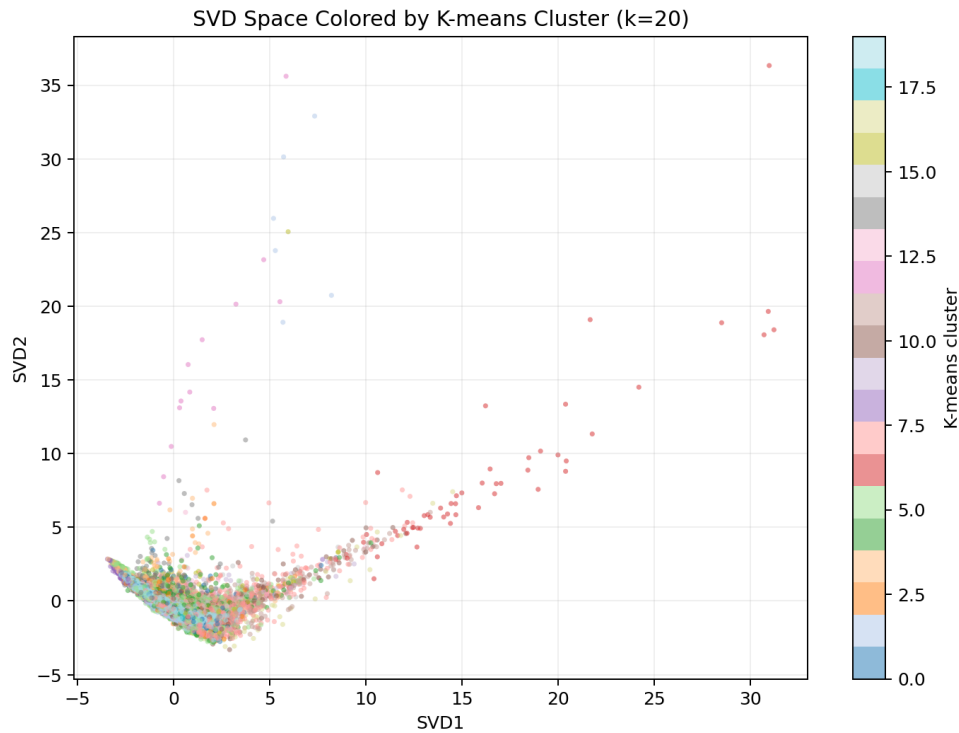


Figure 12. SVD representation colored by selected K-means clusters

Cluster profiles are needed because metrics alone do not give domain meaning. The heatmap checks whether

each cluster has interpretable genre concentration.

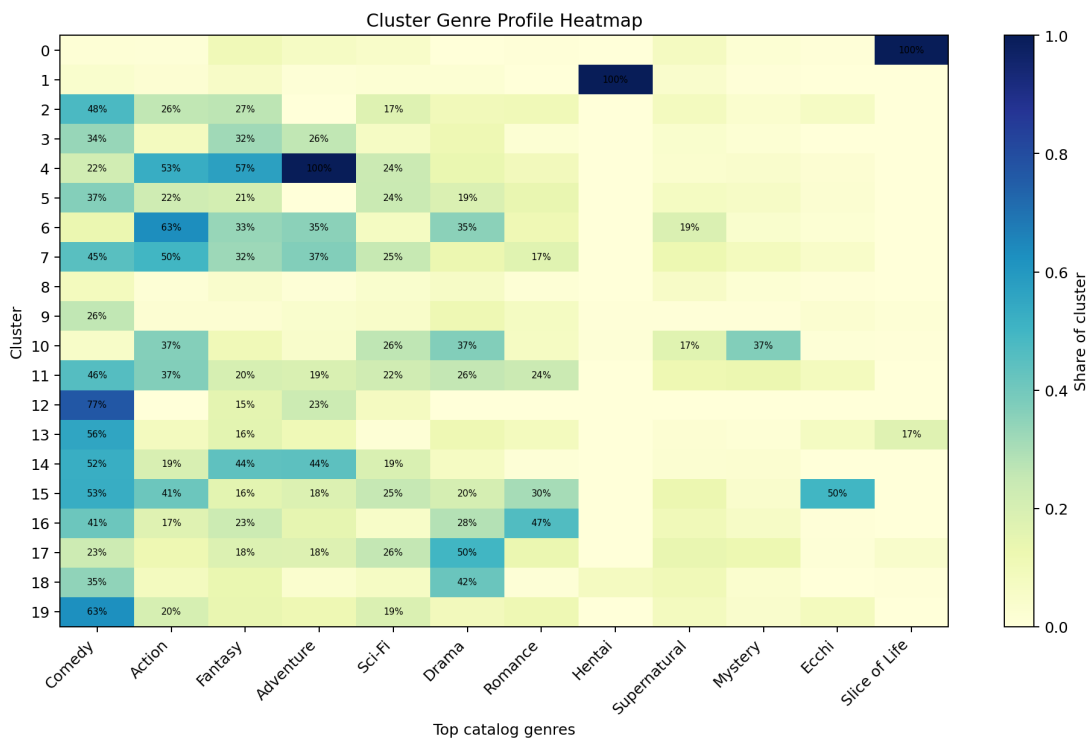


Figure 13. Genre profile heatmap for K-means clusters

DBSCAN and OPTICS are density-based checks. DBSCAN depends on a global ϵ ; OPTICS relaxes that by using reachability ordering, but it still depends on density assumptions. Their higher silhouette values are not directly better than K-means because most points are often labeled as noise. For example, DBSCAN with $\epsilon = 3.0$ and $\min_samples = 35$ reaches silhouette 0.491, but it marks about 90.8 percent of the sample as noise and keeps only two clusters. OPTICS with $\min_samples = 50$ and $\xi = 0.05$ reaches silhouette 0.568, but again labels about 96.5 percent as noise. This is useful failure evidence: anime meta-data contains dense local pockets, but a density method cannot cover the full catalog cleanly without rejecting most titles.

Table 8. Density Method Validation Highlights

Method and setting	Clusters	Silhouette	Noise pct.	Interpretation
DBSCAN $\epsilon = 1.8$, min 10	9	0.351	90.6	many points rejected
DBSCAN $\epsilon = 3.0$, min 20	4	0.421	89.9	compact pockets only
DBSCAN $\epsilon = 3.8$, min 10	20	0.137	60.8	lower noise but weak separation
OPTICS min 10, $\xi = 0.03$	8	0.288	89.1	similar to DBSCAN pockets
OPTICS min 50, $\xi = 0.05$	2	0.568	96.5	high score but too much noise

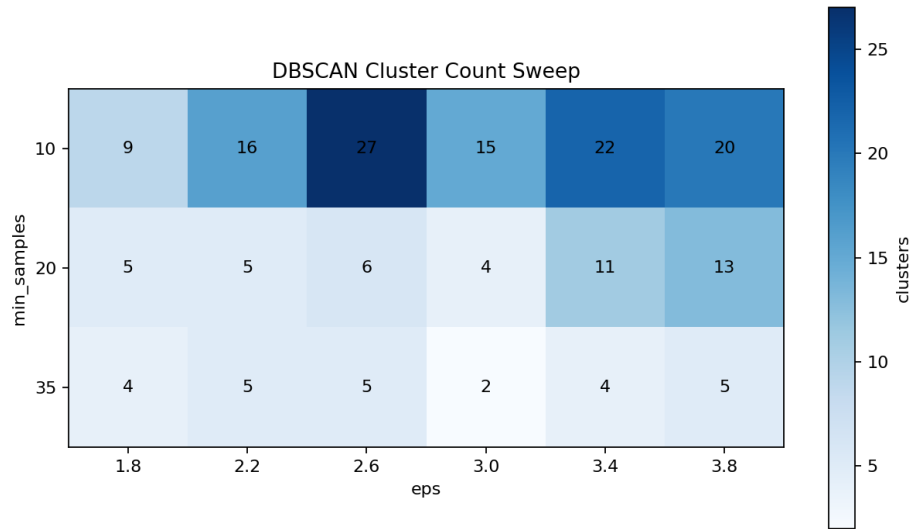


Figure 14. DBSCAN cluster-count sensitivity across eps and min-samples

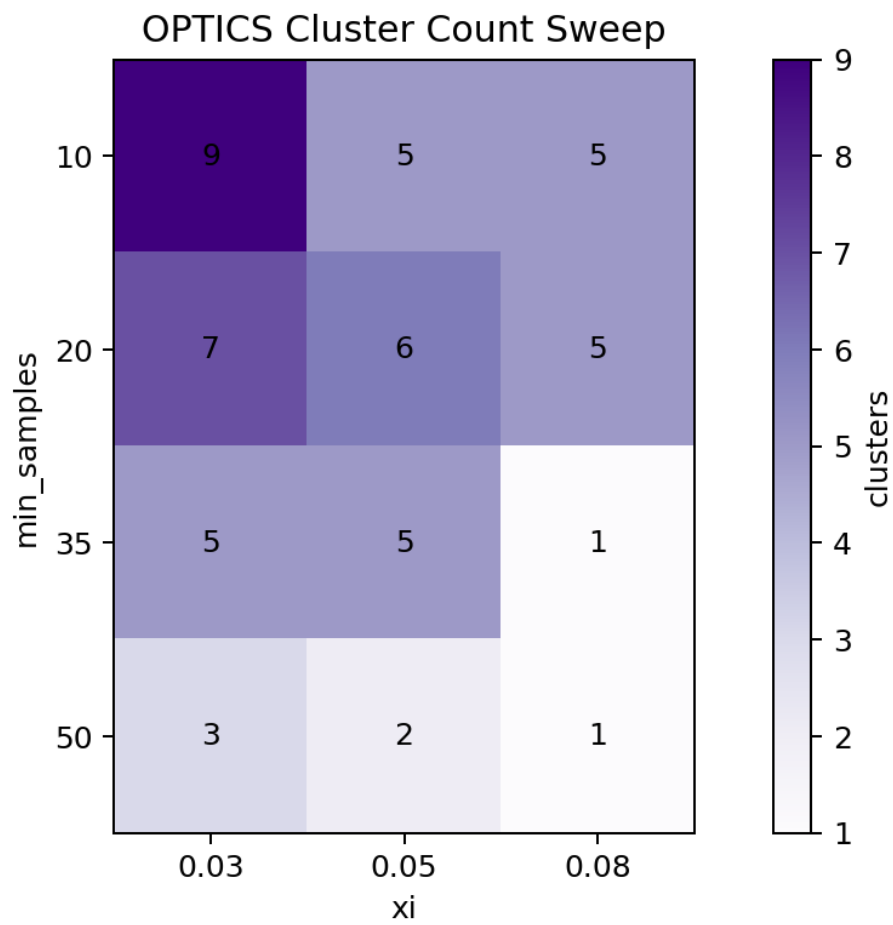


Figure 15. OPTICS cluster-count sensitivity across xi and min-samples

Table 9. Clustering Method Comparison

Method	Assumption	Interpretation in this project
K-means	spherical Euclidean clusters	useful broad segmentation, forced assignment
DBSCAN	global density threshold	finds noise but sensitive to eps
OPTICS	variable-density reachability	density sensitivity check without one global eps

Conclusion

By Week 7, the project has a reproducible processed anime catalog, an aligned large-scale ratings table, feature matrices, dimensionality-reduction results, clustering sweeps, cluster profiles, and item-item graph diagnostics. The most defensible conclusions are cautious: PCA and SVD expose compressible structure, clusters are useful as discovery segments but not hard genre truth, and relation edges are necessary for franchise-aware recommendation because similarity alone can recommend entries out of context.

The remaining limitations are API incompleteness, imperfect MAL-to-AniDB alignment, demographic missingness that cannot always be inferred, and overlap between anime formats such as movies, specials, and split releases. These limitations are visible in the notebooks and build logs, which is important for the final defense.

References

- AniDB Community. 2025. *AniDB HTTP API Specification*. https://wiki.anidb.net/HTTP_API_Definition
- Jikan Developers. 2026. *Jikan API: Unofficial MyAnimeList API*. <https://docs.api.jikan.moe/>
- MyAnimeList. 2025. *MyAnimeList*. <https://myanimelist.net>
- purarue. 2025. *MAL ID Cache Dataset*. <https://github.com/purarue/mal-id-cache>
- Shoko Anime. 2025. *Shoko Server AniDB HTTP XML Cache*. https://files.shokoanime.com/files/shoko-server/other/Anime_HTTP.zip
- Shoko Anime Documentation. 2026. *Understanding AniDB Bans: HTTP API*. <https://docs.shokoanime.com/shoko-server/understanding-anidb-bans#http-api>
- Turan, R. 2025. *User AnimeList Dataset*. Kaggle. <https://www.kaggle.com/datasets/ramazanturann/user-animelist-dataset>